Journal des prompts — Allocation d'équipement sur la blockchain (StudioFlow / ReFrame)

Étudiant : Mohamed Yessine Nehdi — ESPRIT, Génie Logiciel
Fonctionnalité : Allocation d'équipement studio enregistrée sur la blockchain (Solidity, Ganache, MetaMask, Hardhat, Ethers.js)
Outils IA utilisés : Claude (Cowork) pour la compréhension et la planification · Claude Code pour l'implémentation

> Ce document rassemble les prompts que j'ai rédigés pendant le projet, à des fins de transparence pour la validation.
> Les prompts sont recopiés **tels que je les ai écrits**, sans correction.

Partie 1 — Compréhension et planification (Claude, Cowork)

Prompt 1 — Comprendre le projet blockchain

Date : 02/11/2026
Pièce jointe : `ReFrame_Blockchain_Explication_Debutant.docx`

```
im going to make a smat contrat in the project reframe and we are going to use node ganache and metamask for the smart contract in blockchaine the reframe project is already done , i want you to make me understand the what are we going to do , dont change nothing for now just give me what are we going to do
```

Objectif : comprendre le rôle de chaque outil (Node.js, Ganache, MetaMask, Hardhat, Ethers.js) et les étapes globales, sans modifier le code.
Résultat : explication du flux complet (hash du PDF → smart contract → vérification), rôle de chaque outil, étapes et points d'attention.

Prompt 2 — Définir le projet d'allocation d'équipement

Date : 03/11/2026

```
okay lets make a program of making the one project of blockchain that where to aloue the equipments
```

Objectif : définir un projet blockchain pour l'allocation du matériel du studio.
Résultat : analyse du module matériel existant, conception du smart contract `EquipmentAllocation.sol` (fonctions, règles, statuts), plan en 6 phases.

Prompt 3 — Répartition du travail

Date : 03/11/2026

```
okay yosser will do the smart conract and i will do this equipmment allocation
```

Objectif : répartir le travail : Yosser fait le registre des contrats (`ContractRegistry`), je fais l'allocation d'équipement.
Résultat : liste des éléments à partager (projet Hardhat, réseau Ganache, wallet admin, variables d'environnement, branches Git) et plan de ma partie.

Prompt 4 — Explication du projet d'allocation

Date : 04/11/2026

```
okay i want you to explain what we will do and what is this project of the equipment allocation
```

Objectif : comprendre le projet simplement avant de commencer.
Résultat : explication du problème (historique modifiable dans MongoDB), de la solution (historique infalsifiable sur la blockchain), des règles du contrat, du rôle des outils, du scénario de démo et de la séparation MongoDB / blockchain.

Prompt 5 — Contrainte : nouvelle page, sans toucher au travail des collègues

Date : 04/11/2026

```
okay for you to know i want a new page where the photographer want to make an allocation i dont
want to touch the work of my collages
```

Objectif : isoler mon travail dans un module et une page à moi.
Résultat : structure des nouveaux fichiers (`src/modules/allocation/`, `/dashboard/allocation`, projet Hardhat séparé), lecture seule du modèle `Materiel`, collection MongoDB dédiée.

Prompt 6 — Rédiger le prompt pour Claude Code

Date : 04/11/2026

```
okay good now lets make a good prompt for claude code
```

Objectif : obtenir un prompt complet et structuré pour l'implémentation avec Claude Code.
Résultat : le prompt de la Partie 2 ci-dessous.

Partie 2 — Prompt d'implémentation donné à Claude Code

Ce prompt a été préparé à la suite du prompt 6, puis donné à Claude Code. Il fixe les règles (ne pas modifier les fichiers des collègues), la spécification du smart contract et un travail en 4 phases avec validation à chaque étape.

```
# Task: Blockchain equipment allocation for StudioFlow (my own module — new files only)

## Context
StudioFlow is a Next.js 16 (App Router) + MongoDB/Mongoose 9 app for photo studios, built by a
team.
I am adding a NEW feature: a page where a photographer allocates studio equipment, and every
allocation step is recorded on a local Ethereum blockchain (Ganache) through a Solidity smart
contract, signed with MetaMask.

Goal: a tamper-proof history of who used which equipment, when, and in what condition it came
back.

## STRICT RULES — read before anything else
1. **Do NOT modify any existing file written by my teammates.** That includes everything in
   `src/modules/materiel/`, `src/modules/seances/`, other modules,
`src/app/(backoffice)/dashboard/materiel/`,
   shared components, `nav-config.ts`, `.env.example`, `tsconfig.json`, configs, etc.
2. I may only CREATE new files in:
   - `src/modules/allocation/`
   - `src/app/(backoffice)/dashboard/allocation/`
   - the new Hardhat project `../StudioFlow-Blockchain/` (sibling folder of the repo, NOT
inside it)
3. The only allowed change to a shared file is adding the `ethers` dependency to
`package.json`.
   **Ask me before doing it.** If any other shared file seems necessary, STOP and ask me.
4. The existing `Materiel` model may be **imported and read only** (list equipment). Never
write to it,
   never write to `reservations_materiel`.
5. No personal data on the blockchain: only hashed IDs, dates, statuses, conditions, wallet
addresses.
6. Never put a private key in frontend code or commit one. Transactions are signed by MetaMask
in the browser.

## Before coding
- Read `AGENTS.md`, `README.md`, and `src/modules/materiel/` (types, rules, schemas, service,
actions,
  components) to copy the project's conventions: Zod schemas, Server Actions returning
`ActionResult`,
  `StatusConfig` + status badges, shadcn/ui components, file naming, "Module : … · Responsable
: …" headers
  (use "Module : Allocation Blockchain · Responsable : Mohamed Yessine Nehdi").
- Reuse the existing reservation rules logic by **importing** `seChevauchent` / `peutPasser`
from
  `reservation-materiel.rules.ts` if helpful (import only, no edits).
- Then give me a short plan and wait for my OK.

## Tech stack
- Node.js, Ganache (`npx ganache`, RPC http://127.0.0.1:8545, chain id 1337, fixed mnemonic so
accounts are stable)
- Hardhat 3 (TypeScript, Mocha + Ethers.js, Hardhat Ignition), Solidity ^0.8.24
- Ethers.js v6 in the Next.js app (`BrowserProvider` for MetaMask, `JsonRpcProvider` on the
server)
- MetaMask with 2 imported Ganache accounts: #0 = Admin (contract owner), #1 = Photographer

---
```

```
## PHASE 1 — Hardhat project + smart contract (stop after this phase)
Create `../StudioFlow-Blockchain/` with a `ganache` network in `hardhat.config.ts`
(Hardhat 3 syntax: `type: "http"`, url, accounts from env — never hardcoded keys).

Write `contracts/EquipmentAllocation.sol`:

**State**
- `address public owner` (set in constructor)
- `mapping(address => bool) public staff` — photographers allowed to book
- `enum Condition { New, Good, Worn, OutOfOrder }`
- `enum Status { None, Pending, Confirmed, InUse, Returned, Cancelled }`
- `struct Equipment { Condition condition; bool active; bool exists; }`
- `struct Allocation { bytes32 equipmentId; bytes32 seanceId; address bookedBy; uint64 start;
uint64 end; Status status; Condition returnCondition; uint64 createdAt; }`
- `mapping(bytes32 => Equipment) equipments`, `mapping(bytes32 => Allocation) allocations`,
  `mapping(bytes32 => bytes32[]) equipmentAllocations` (history per item)
- IDs = `keccak256(bytes(mongoObjectIdString))`, computed off-chain.

**Functions** (each emits an event; use custom errors)
| Function | Caller | Checks |
|---|---|---|
| `addStaff(address)` / `removeStaff(address)` | owner | — |
| `registerEquipment(bytes32 id, Condition c)` | owner | not already registered |
| `requestAllocation(bytes32 allocId, bytes32 equipId, bytes32 seanceId, uint64 start, uint64
end)` | staff or owner | allocId unused, equipment exists + active + not OutOfOrder, end >
start, **no overlap** with that item's Pending/Confirmed/InUse allocations (A.start < B.end &&
B.start < A.end; touching is OK) |
| `confirmAllocation(allocId)` | owner | Pending → Confirmed |
| `checkout(allocId)` | owner or bookedBy | Confirmed → InUse |
| `returnEquipment(allocId, Condition c)` | owner | InUse → Returned; c must be Good or Worn;
updates equipment condition |
| `cancelAllocation(allocId)` | owner or bookedBy | Pending or Confirmed → Cancelled |
| `reportBreakdown(equipId)` | staff or owner | sets OutOfOrder |
| `isAvailable(equipId, start, end) view`, `getAllocation(allocId) view`,
`getEquipmentHistory(equipId) view` | anyone | — |

Transitions must match `TRANSITIONS_RESERVATION` in the materiel module.

Write `test/EquipmentAllocation.ts` covering at least: deploy/owner, staff management, register
equipment,
happy path (request → confirm → checkout → return), overlap refused, touching periods allowed,
non-staff refused, non-owner confirm refused, invalid transitions refused, cancel by
booker/owner,
broken equipment can't be booked, return updates condition, events emitted, history returned.
Run `npx hardhat test` — all green.

Ignition module `ignition/modules/EquipmentAllocation.ts` + a script that, after deploying to
Ganache,
writes `{ address, chainId, abi }` to
`src/modules/allocation/contract/EquipmentAllocation.json` in the app.
Add a `README.md` in the Hardhat project with exact commands (start Ganache, test, deploy,
MetaMask setup).

➡️ **Stop and show me: test output, contract summary, how to deploy.**

## PHASE 2 — App module `src/modules/allocation/` (stop after this phase)
- `allocation.types.ts` — types, status/condition labels + `StatusConfig` (same style as
materiel)
- `allocation.model.ts` — NEW collection `allocations_blockchain`:
  materiel ref, seance id + title copy, utilisateur ref, walletAddress, dateDebut, dateFin,
statut,
  etatRetour, chainAllocationId (bytes32), txHashes (array: { action, txHash, blockNumber, at
}), timestamps.
- `wallet-link.model.ts` — NEW collection `wallets_utilisateurs` (userId ↔ wallet address).
  Do NOT edit the Utilisateur model.
- `allocation.schemas.ts` — Zod.
```

```
- `blockchain.server.ts` — server-only: `JsonRpcProvider`, read contract, and **verify a
txHash**
  (receipt exists, status 1, `to` = contract address, expected event present with matching
allocId)
  before saving anything to MongoDB. Never trust a txHash sent by the client without checking.
- `blockchain.client.ts` — client: connect MetaMask, check/switch to chain 1337
(`wallet_switchEthereumChain`,
  fallback `wallet_addEthereumChain`), get signer, call contract functions, return txHash.
- `allocation.service.ts` + `allocation.actions.ts` — list my allocations, list all (admin),
get by id
  with on-chain history, save after verified tx, link wallet. Respect existing auth/roles the
way the
  materiel module does.
- Flow: server creates the Mongo ObjectId first → client computes allocId = keccak256 →
MetaMask signs →
  server action verifies tx → saves to Mongo. If the tx fails/is rejected, nothing is saved.
- Admin helper action: register an equipment item on-chain (owner wallet) + add a photographer
wallet as staff.

➡️ **Stop and show me the files and the flow.**

## PHASE 3 — Pages `src/app/(backoffice)/dashboard/allocation/` (stop after this phase)
- `page.tsx`:
  - "Connect MetaMask" button (shows short address, wrong-network warning, MetaMask-not-
installed message)
  - Allocation form: equipment (read from Materiel, non-archived, available), séance, start/end
dates,
    live "available on blockchain?" check via `isAvailable`
  - "Allocate" → MetaMask signature → success toast with tx hash
  - "My allocations" table with status badges and ✅ "On blockchain" badge
  - Admin view (same page, by role): all allocations + buttons Confirm / Mark as taken / Record
return
    (condition Good/Worn) / Cancel, plus "Register equipment on-chain" and "Add photographer
wallet"
- `[id]/page.tsx`: allocation details + **Blockchain history** timeline (action, wallet, date,
block, tx hash)
  read from the contract events, compared with MongoDB (show a warning if they differ).
- Clear error messages: MetaMask rejected, Ganache offline, wrong network, overlap, not staff.
- No link in `nav-config.ts` (shared file): the page is reachable at `/dashboard/allocation`.

➡️ **Stop and show me screenshots or how to run it.**

## PHASE 4 — Checks
- `npx hardhat test` green, app `npm run lint` and type-check pass, existing tests still pass.
- `git status` must show **no modified files outside** my allowed folders (except
`package.json` /
  `package-lock.json` for `ethers`, if I approved). List the new files.
- Write `src/modules/allocation/README.md`: architecture, env vars to add in `.env.local`
  (`NEXT_PUBLIC_CHAIN_ID=1337`, `NEXT_PUBLIC_RPC_URL=http://127.0.0.1:8545`,
`GANACHE_RPC_URL`),
  full demo scenario (photographer books → admin confirms → checkout → return → history).

## Working style
- Work phase by phase; at the end of each phase, summarise what you did and wait for my OK.
- Explain blockchain-specific choices briefly (I'm learning).
- If anything is ambiguous or would touch a teammate's file, ask instead of guessing.
```

Partie 3 — Prompts envoyés à Claude Code pendant l'implémentation

> Note : les premiers prompts d'implémentation (phases 1 à 4, puis la mise en place de Docker) seront ajoutés à partir de l'historique complet de Claude Code.

Prompt CC-1 — Erreurs Docker et Ganache

Date : 12/11/2026
Contexte : après la mise en place de Docker (app + MongoDB + Ganache lancés avec `docker compose up -d`), j'ai collé la sortie de mon terminal.

```
mohamedyessinenahdi@Mohameds-MacBook-Air StudioFlow-Blockchain % docker compose up
no configuration file provided: not found
mohamedyessinenahdi@Mohameds-MacBook-Air StudioFlow-Blockchain % npm run deploy
> hardhat ignition deploy ignition/modules/EquipmentAllocation.ts --network ganache
Error HHE703: Cannot connect to the network "ganache".
mohamedyessinenahdi@Mohameds-MacBook-Air StudioFlow-Blockchain % npm run export
> hardhat run scripts/export-to-app.ts --network ganache
Error HHE703: Cannot connect to the network "ganache".
```

Objectif : comprendre et corriger les erreurs.
Résultat :

- **Cause :** `docker compose up` avait été lancé depuis `StudioFlow-Blockchain`, alors que `docker-compose.yml` se trouve dans `StudioFlow`. Ganache n'était donc pas démarré, ce qui faisait échouer `deploy` et `export` (rien n'écoutait sur le port 8545).
- **Correction :** Claude Code a relancé la stack depuis le bon dossier. Les 3 services tournent (app, ganache, mongo), le contrat est toujours déployé à la même adresse et la chaîne est au bloc 30 : l'historique est conservé.
- **Routine retenue :** `cd ~/Desktop/StudioFlow/StudioFlow`, puis `docker compose up -d` pour démarrer et `docker compose stop` pour arrêter. `npm run deploy && npm run export` ne servent qu'en cas de modification du contrat ou de réinitialisation de la chaîne.

Prompt CC-2 — Ouvrir le projet blockchain

Date : 12/11/2026

```
broo if i wana open the blockchain project how to do it
```

Objectif : savoir où se trouve le projet Hardhat et comment l'ouvrir.
Résultat :

- Le projet est un dossier séparé : `~/Desktop/StudioFlow/StudioFlow-Blockchain`.
- Trois façons de l'ouvrir : `code ~/Desktop/StudioFlow/StudioFlow-Blockchain` (nouvelle fenêtre VS Code), *File → Add Folder to Workspace* (à côté de l'app), ou `cd` dans le dossier

puis `npm test`.

- Fichiers importants : `contracts/EquipmentAllocation.sol` (le contrat), `test/EquipmentAllocation.ts` (les tests), `scripts/export-to-app.ts` (copie l'adresse et l'ABI dans l'app), `README.md`.
- À éviter : lancer `npm run ganache` pendant que Docker tourne, car les deux utilisent le port 8545.

Prompt CC-3 — Ouvrir l'application

Date : 12/11/2026

```
okay how to open the app
```

Objectif : lancer et utiliser l'application.
Résultat :

1. Ouvrir Docker Desktop.
2. `cd ~/Desktop/StudioFlow/StudioFlow` puis `docker compose up -d`.
3. Ouvrir http://localhost:3000 dans le navigateur où MetaMask est installé.
4. Se connecter en admin ou en photographe.
5. Aller dans **Blockchain → Equipment allocation**.

Pour modifier le code : `code ~/Desktop/StudioFlow/StudioFlow`, puis `docker compose up --build -d` pour voir les changements.

Prompt CC-4 — Afficher les tests réalisés

Date : 13/11/2026

```
show me the tests we did
```

Objectif : montrer les tests et vérifier qu'ils passent.
Résultat : Claude Code a lancé les deux suites de tests : **45 tests, tous réussis.**

1. Tests du smart contract (Hardhat + Mocha) : 30 réussis

Groupe	Ce qui est vérifié
deployment	Le déployeur devient propriétaire (admin).
staff management	Seul l'admin ajoute ou retire des photographes ; l'adresse zéro est refusée.
equipment registration	Un équipement n'est enregistré qu'une fois, et seulement par l'admin.
happy path	Le cycle complet fonctionne : demande → confirmation → sortie → retour.
availability & overlap	Pas de double réservation sur des dates qui se chevauchent ; les réservations bout à bout sont acceptées ; une annulation ou un retour libère les dates.

Groupe	Ce qui est vérifié
permissions	Un wallet non autorisé ne peut pas réserver ; seul l'admin confirme ou enregistre un retour ; un photographe ne peut pas toucher à la réservation d'un autre.
transitions	Aucune étape ne peut être sautée (ex. sortie avant confirmation refusée).
cancel	Le photographe annule sa réservation en attente ; l'admin annule une réservation confirmée.
condition	Un équipement en panne ou inactif ne peut pas être réservé ; le retour enregistre l'état « Good » ou « Worn ».
history	L'historique complet d'un équipement est relu depuis la blockchain.

Commande : `cd ../StudioFlow-Blockchain && npm test`

2. Tests de l'application (Vitest) : 15 réussis

- **8 tests de règles** (sans Ganache) : les IDs sont hachés comme dans le contrat ; les codes d'état et de statut correspondent des deux côtés ; les dates sont bien converties en secondes ; une signature « lier mon wallet » n'est valable que 10 minutes et pour le bon compte.
- **7 tests blockchain** (avec Ganache, transactions réelles) : le serveur accepte une vraie transaction mais refuse un hash de transaction si un autre wallet le présente, s'il concerne une autre réservation ou d'autres dates, s'il est présenté comme preuve d'une autre étape, s'il n'a pas été envoyé à notre contrat, ou s'il n'existe pas.

Commande : `npx vitest run --config src/modules/allocation/tests/vitest.config.ts`
(Sans Ganache, les 7 tests blockchain sont ignorés, pas en échec.)

Prompt CC-5 — Générer le fichier des prompts

Date : 13/11/2026

```
give me all the prompts and what is the output of evrey prompt in this convo i want a file
```

Objectif : produire un fichier avec tous les prompts de la conversation Claude Code et leurs résultats, pour la validation.
Résultat : Claude Code a commencé à lire l'historique de la conversation pour générer ce fichier.